AI POWERED SAST AGENT

A Major Project Report Submitted in Partial Fulfillment of the Requirements for the

Award of the Degree of

BS-MS (Computer Science) – 5 yrs Integrated

By

Sai Sanjana Gujjari
217022026028

Dumpala Pardha Saradhi
217022026007

Under the Supervision of

Ms. Roshini Chinthala
Assistant Professor, Dept. of Computer Science

VVISM

OSMANIA UNIVERSITY

Hyderabad

2025 – 2026

SECNODE

www.secnode.ai

Date: July 4, 2026

PROJECT COMPLETION CERTIFICATE

We would like to certify that the following student of **BS-MS (Computer Science) – 5 Year Integrated Program** from **Vishwa Vishwani Institute of Systems and Management** has successfully completed her project at **SECNODE Pvt. Ltd.**

Gujjari Sai Sanjana

We state on record that she has successfully completed the project titled **AI Powered SAST**, which focused on developing intelligent static application security testing solutions for automated vulnerability detection. During the project, she worked closely with the development team, demonstrated strong technical skills, determination, and creativity, and completed the work before the deadline.

Throughout the engagement, she adhered to company guidelines and maintained confidentiality of all project-related information.

We wish her all the best in her future endeavours and hope she continues to display the same commitment and excellence.

Sincerely,

Vishnu
Technical Head
SECNODE Pvt. Ltd.

SECNODE www.secnode.ai

Date: July 4, 2026

PROJECT COMPLETION CERTIFICATE

We would like to certify that the following student of **BS-MS (Computer Science) – 5 Year Integrated Program** from **Vishwa Vishwani Institute of Systems and Management** has successfully completed his project at **SECNODE Pvt. Ltd.**

Dumpala Pardha Saradhi

We state on record that he has successfully completed the project titled **AI Powered SAST**, which focused on developing intelligent static application security testing solutions for automated vulnerability detection. During the project, he worked closely with the development team, demonstrated strong technical skills, determination, and creativity, and completed the work before the deadline.

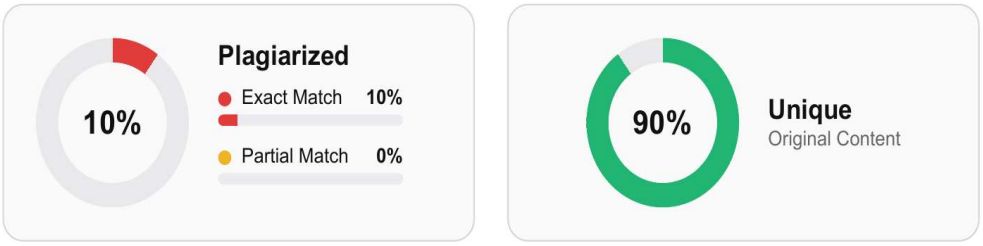
Throughout the engagement, he adhered to company guidelines and maintained confidentiality of all project-related information.

We wish him all the best in his future endeavours and hope he continues to display the same commitment and excellence.

Sincerely,

Vishnu
Technical Head
SECNODE Pvt. Ltd.

PLAGIARISM VERIFICATION CERTIFICATE



Plagiarism detection tool	DupliChecker
Similarity index	10% (Exact 10% · Partial 0%)
Unique / original content	90%
Date of verification	July 4, 2026
Report source	https://www.duplichecker.com/

Similarity index recorded from the DupliChecker report · https://www.duplichecker.com/

ACKNOWLEDGEMENT

This report is a product of not only our sincere efforts but also the guidance and support given by our teachers and mentors. We take this opportunity to express our gratitude to all who have been instrumental in the successful completion of this project.

We would like to express our gratitude to VVISM and to all the people who extended unending support at all the stages of the project.

We would like to express our profound gratitude to **Ms. Roshini Chinthala, Assistant Professor**, for the invaluable guidance, monitoring and constant encouragement throughout the project work.

We are thankful to **Dr. P. Chakravarthi Sir**, Director, BS-MS (CS) and **Mr. Ch. Mahesh Kumar Sir**, Vice Principal, for providing us the opportunity to undergo this project. Their valuable support in providing excellent infrastructure and a conducive environment helped us in completing the work smoothly as per the given schedule.

Special thanks to Secnode for granting me access to essential data and resources. We are obliged for their cooperation and supervision provided throughout the course of this project.

We thank all the teaching, administrative and non-teaching members of VVISM for their constant support and help.

Lastly, we are indebted to our families for their unwavering moral support during this journey, without which this project work would not have been possible. We sincerely acknowledge and thank all those who gave support directly and indirectly in the completion of this project.

Sai Sanjana Gujjari

217022026028

Dumpala Pardha Saradhi

217022026007

VISHWA VISHWANI
INSTITUTE OF SYSTEMS & MANAGEMENT

DECLARATION

We hereby declare that the Major Project Report entitled **AI POWERED SAST AGENT** submitted by us in partial fulfilment for the Degree of BS-MS (Computer Science) – 5 yrs Integrated, IV Year / VIII Sem to the Osmania University, Hyderabad, is a bonafide work undertaken by us. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Sai Sanjana Gujjari
Reg No: 217022026028

Dumpala Pardha Saradhi
Reg No: 217022026007

Place: Hyderabad

Signature of the Students

Date:

ABSTRACT

Conventional rule-based Static Application Security Testing (SAST) solutions depend on hand-crafted vulnerability pattern libraries that generate voluminous technical reports, driving alert fatigue and leaving engineering teams without clear remediation guidance. This project — the AI Powered SAST Agent — addresses these shortcomings by constructing a full-stack web platform in which Large Language Models (LLMs) serve as the primary scanning engine, directly reading and reasoning over source code to detect, classify, and explain security weaknesses without relying on any external binary tool.

Developers submit a GitHub repository URL; the Ai Scanner Service recursively traverses the codebase, dispatches each source file to a resilient multi-provider LLM pipeline, and obtains structured JSON findings that include severity rating, precise line numbers, CWE and OWASP identifiers, a plain-English explanation, a proof-of-concept attack scenario, and a ready-to-apply code fix — all surfaced live on the dashboard through a Server-Sent Events stream.

The platform is built with Node.js and Express on TypeScript, a PostgreSQL database managed through Prisma ORM, and a React 18 dashboard. The AI engine combines NVIDIA NIM (Llama 3.1-70B) for low-latency inference, Google Gemini 1.5 Flash as the primary analysis engine, and OpenAI GPT-4o-mini as an automatic fallback. Findings are streamed live with per-issue AI Insight Panel.

This project demonstrates that combining AI-native scanning, real-time progress streaming, and contextual per-finding remediation yields a transparent, developer-friendly alternative to traditional rule engines. The entire stack is containerised with Docker Compose.

LIST OF FIGURES

LIST OF TABLES

LIST OF SCREENSHOTS

TABLE OF CONTENTS

1. INTRODUCTION

Application security vulnerabilities pose a significant threat to modern software systems. While Static Application Security Testing (SAST) tools have existed for decades, their reliance on curated rule libraries produces verbose reports that overwhelm developers, leading to alert fatigue and ignored vulnerabilities. The primary goal of this project is to replace rule-based scanning with AI-native analysis using Large Language Models that directly read, reason about, and explain security vulnerabilities in source code.

Traditional SAST tools fail to detect context-aware vulnerabilities and provide no explanation or remediation guidance. The AI Powered SAST Agent is an automated platform that identifies security issues through deep semantic reasoning over source code, rather than matching entries in a rule library.

1.1 Background and Domain Overview

Software security incidents cost the global economy an estimated USD 8 trillion in 2023, with a substantial share traced to weaknesses that were detectable during the development phase. Established SAST tools such as SonarQube, Semgrep, and Checkmarx build their detectors around curated rule libraries. Although effective against well-catalogued vulnerability patterns, these tools struggle to identify context-dependent or logic-level flaws, and their output — a list of rule identifiers and line references — provides no guidance on exploitability, business impact, or how to remediate the finding.

Modern development teams require tools that not only detect issues but also guide developers through understanding and fixing them. The cost of fixing a vulnerability discovered in production is many times higher than fixing the same vulnerability during development. A tool that provides actionable, context-rich findings during the development phase therefore has substantial value both economically and from a security-posture perspective.

1.2 AI-Native Scanning vs Rule-Based SAST

The emergence of Large Language Models such as Google Gemini, NVIDIA Llama 3.1, and OpenAI GPT-4 has created an opportunity to replace rule-based scanning entirely. LLMs possess deep semantic understanding of programming languages, security vulnerability patterns, and remediation strategies. The AI Powered SAST Agent uses LLMs as the primary scanning engine — not merely as an enrichment layer over an existing scanner — so the AI itself discovers, classifies, explains, and suggests fixes for vulnerabilities in a single unified pipeline.

Unlike rule engines, an LLM does not need a pre-written pattern for every vulnerability class. By reasoning over the entire context of a source file, the model can identify logic-level flaws such as insecure authorisation checks, broken access control, and unsafe data flow that rule engines typically miss.

1.3 Multi-LLM Pipeline

To ensure maximum reliability and performance, the platform implements a resilient multi-provider LLM pipeline. NVIDIA NIM (Llama 3.1) is used for ultra-fast, low-latency inference. Google Gemini 1.5 Flash serves as the primary analysis engine for high-speed large-batch processing. OpenAI GPT-4o-mini acts as a reliable fallback when rate limits or provider failures occur. The system automatically routes requests through the chain, ensuring scans always complete successfully even when an individual provider is unavailable.

1.4 System Architecture Overview

The platform is built on a production-grade technology stack: Node.js with Express and TypeScript for the backend API, React 18 with Vite and Tailwind CSS for the frontend dashboard, PostgreSQL with Prisma ORM for data persistence, Redis as the backing store for the Bull scan job queue, and Docker Compose for fully containerised deployment. GitHub OAuth provides secure, passwordless authentication; identity is thereafter maintained via JWT tokens stored in HTTP-only cookies and validated by a Passport JWT strategy.

1.5 Existing System and its Drawbacks

Current SAST solutions such as SonarQube, Semgrep, and Checkmarx are built on curated rule libraries that match known vulnerability patterns against source code. While effective at detecting well-defined issues, these tools depend on security engineers to write and maintain rules for every new vulnerability class or programming framework. Raw scanner output is presented as technical identifiers with no contextual explanation, impact assessment, or fix suggestion, leaving developers without the guidance needed to act. Most tools are CLI-only, requiring deep security expertise to operate, and provide no unified platform for scanning, enrichment, and historical tracking in a single interface.

The traditional SAST mechanisms suffer from the following critical limitations:

- Rule-based engines are limited to known vulnerability patterns; logic-level and context-aware flaws go undetected. New vulnerabilities require continuous, expensive rule authoring.
- High false-positive rates cause alert fatigue, which leads developers to ignore warnings entirely.
- No actionable remediation is provided. Developers receive vulnerability identifiers but not plain-English explanations, impact assessments, or code-level fix suggestions.
- Rule libraries require continuous manual maintenance for every new language, framework, or vulnerability class.
- Most tools require command-line operation and security expertise to configure and interpret, excluding the majority of development teams.

1.6 Proposed System

The AI Powered SAST Agent eliminates rule-based scanning by positioning Large Language Models as the primary analysis engine. The AiScannerService dispatches every source file to a resilient multi-provider pipeline — NVIDIA NIM (Llama 3.1-70B) for low-latency inference, Google Gemini 1.5 Flash as the primary engine, and OpenAI GPT-4o-mini as an automatic fallback on failure — which returns structured JSON findings. AI enrichment fields (explanation, impact, proof-of-concept, remediation, and fix code) are written directly onto each Issue record in PostgreSQL. Findings reach a React dashboard in real time through an SSE stream that shows severity distribution charts, a Monaco Editor code viewer with line-level highlighting, and a per-issue AI Insight Panel. All scans are retained in the database for historical comparison and security-posture tracking.

The proposed system aims to overcome the limitations of existing tools through the following key capabilities:

- AI-Native Scanning: the AiScannerService submits each source file directly to an LLM with a structured security audit prompt, receiving findings as structured JSON without any external binary dependency.
- Multi-LLM Resilience: the NVIDIA NIM → Gemini 1.5 Flash → GPT-4o-mini fallback chain ensures maximum availability under load.
- AI Enrichment per Finding: every vulnerability is automatically enriched with a plain-English explanation, contextual impact assessment, proof-of-concept attack scenario, CWE/OWASP mapping, and a ready-to-apply code fix.
- Real-Time Dashboard: findings are streamed live via SSE to a React dashboard with severity charts, a Monaco Editor code viewer, and an AI Insight Panel.
- Scan History and Comparison: all scans are persisted in PostgreSQL for side-by-side comparison to track security posture improvements over time.

1.7 Advantages of the Developed System

- AI-native scanning eliminates rule-maintenance overhead and detects context-aware, logic-level vulnerabilities that rule-based tools miss.
- Every finding includes a plain-English explanation, proof-of-concept, and ready-to-apply fix, making security actionable for developers of all experience levels.
- Real-time SSE streaming delivers live scan progress and findings to the dashboard as they are discovered.
- Since the model reasons over code semantics rather than matching rule entries, it can detect vulnerabilities not covered by any existing rule library.

2. REVIEW OF LITERATURE

This chapter discusses the software engineering principles, analysis and design concepts, and methodology used in the development of the AI Powered SAST Agent. These concepts collectively support the development of a robust, scalable, and maintainable platform.

2.1 Description of the Software Engineering Concepts Used

- Modular Service Architecture: the system is divided into independent TypeScript modules — AiScannerService, AIEnrichmentService, ScanProcessor, and REST controllers — each with distinct responsibilities, enhancing maintainability and testability.
- User-Centred Design: the React dashboard is designed around the developer workflow: submit scan → monitor progress → view findings → understand impact → apply fix, in a single linear interface that requires no security expertise.
- Separation of Concerns: Express REST controllers handle HTTP routing; service classes contain business logic; Prisma ORM handles all database interactions; the AI layer is fully abstracted behind a provider-agnostic interface.
- Agile Iterative Development: the system was built iteratively — authentication, then scan submission, then the AI scanning engine, then enrichment, then the dashboard — with each layer tested before the next was added.
- Resilience Engineering: the multi-LLM fallback pipeline, retry logic, rate limiting, and Docker containerisation all follow fault-tolerant design principles, ensuring that the system remains operational under adverse conditions.

2.2 Description of the Analysis and Design Concepts Used

- **Data Flow Diagrams (DFD):** used at Level 0 (system context) and Level 1 (module decomposition) to model how source code, LLM prompts, JSON findings, and AI insights flow through the platform.
- **Object-Oriented Analysis and Design (OOAD):** the Prisma schema models (User, Scan, Issue) represent real-world entities with typed relationships, encapsulating attributes and access patterns.
- **Use Case Diagrams:** define the functional scope from each actor perspective — the Developer submitting scans, receiving enrichment, and comparing history; the System managing LLM routing, file cleanup, and SSE streaming.
- **Activity Diagrams:** model the complete scan lifecycle from submission through AI scanning, enrichment, and completion notification.
- **State Machine Design:** the Scan lifecycle (PENDING → RUNNING → COMPLETED/FAILED) and Issue lifecycle (discovered → enriched → reviewed) are formally modelled as state machines guiding conditional logic.

2.3 Description of the Methodology Used

The development of the AI Powered SAST Agent followed a full-stack web engineering methodology with AI integration, involving the following key steps:

- **Data Collection:** the system scans real-world open-source repositories to validate AI findings against known CVEs, ensuring the accuracy of the LLM-based detection approach.
- **System Design:** designed the Express REST API (14 endpoints across three domain modules), the PostgreSQL schema (three models: User, Scan, Issue), the multi-LLM provider routing logic with automatic fallback, the SSE streaming architecture, and the Docker Compose service topology.

- **Implementation:** built authentication first, then scan submission, then the recursive file walker and LLM audit pipeline, then AI enrichment, and finally the React dashboard.
- **Testing:** applied unit testing (Vitest), integration testing (Supertest), API testing (Bruno), security testing, and performance testing across all modules.
- **Deployment:** packaged all services (API, frontend, PostgreSQL, Redis) in Docker Compose with an Nginx reverse proxy for TLS termination and static asset serving.
- **User Interface and Backend:** a React-based web interface serves all developer workflows, while the backend handles LLM routing, result persistence, and SSE streaming.

3. SOFTWARE REQUIREMENT ANALYSIS

This chapter presents the modular structure of the proposed system, its functional and non-functional requirements, and a description of the overall process. The aim is to define what the system must do, the constraints under which it operates, and how its various components interact.

3.1 Description of Modules

The proposed system is structured into six modules, each providing a distinct functionality. Every module is designed to interact with the PostgreSQL database for security scan management.

3.1.1 Authentication Module

Description: handles all user identity and access management for the platform.

Key Operations:

- GitHub OAuth 2.0 login flow via Passport.js — no passwords are stored.
- JWT tokens, signed on GitHub OAuth callback completion, are stored in an HTTP-only cookie and validated by the Passport JWT strategy on every protected route; no server-side session store is required for authentication.
- User profile is stored in PostgreSQL (id, githubId, email, name, avatarUrl).

3.1.2 Scan Submission Module

Description: accepts source code from two input methods and initiates the scan pipeline.

Key Operations:

- GitHub URL input: validates format and clones the repository using git clone --depth=1.
- Creates a Scan record in PENDING state and returns HTTP 202 with scanId for SSE subscription.

3.1.3 AI Scan Engine Module

Description: the core analysis engine that replaces all rule-based scanning binaries. It uses LLMs directly to discover security vulnerabilities in source code.

Key Operations:

- AiScannerService performs a recursive directory walk of the repository.
- Identifies relevant source files by extension (.ts, .tsx, .js, .jsx, .py, .go, .java, .rb, .php, .c, .cpp, .h); skips files exceeding 60 KB and ignores directories such as node_modules, .git, dist, build, vendor, coverage, __pycache__, .next, and .nuxt.
- Sends each file to the configured LLM with a structured security audit prompt.
- The LLM returns structured JSON findings: ruleId, title, severity, filePath, lineStart, lineEnd, codeSnippet, cweId, owaspId.
- Supports NVIDIA NIM → Gemini 1.5 Flash → GPT-4o-mini routing with automatic fallback.

3.1.4 Result Processing Module

Description: normalises AI-generated findings and persists them for display and tracking.

- Calculates the risk score using the formula: Critical × 10 + High × 5 + Medium × 2 + Low × 1.
- Updates the Scan record with totalIssues, severity counts, and riskScore.

3.1.5 AI Enrichment Module

Description: provides deep per-issue enrichment including explanation, proof-of-concept, and remediation.

- Selects all CRITICAL and HIGH severity issues where aiProcessed is false, after the scanning phase completes.
- Calls Gemini 1.5 Flash as the primary provider; falls back to GPT-4o-mini on rate-limit errors.
- Writes enrichment data (aiExplanation, aiImpact, aiProofOfConcept, aiRemediation, aiFixCode, aiProcessed, aiProcessedAt) directly onto each Issue record via a Prisma update; processed in concurrent batches of five at a parallelism of two to balance throughput and provider rate limits.

3.1.6 Dashboard and Visualisation Module

Description: presents scan results through interactive charts and detailed issue views.

- Severity distribution pie chart using Recharts (Critical / High / Medium / Low).
- Issue list with severity badge, category tag, and file path display.
- Monaco Editor code viewer with red highlight on vulnerable line numbers.
- AI Insight Panel showing explanation, proof-of-concept, impact, and fix code per issue.
- Real-time scan progress via Server-Sent Events stream.

3.2 Functional Requirements

The functional requirements of the system are summarised in Table 3.1.

FR ID	Requirement
FR1	Users can authenticate securely via GitHub OAuth 2.0 without storing passwords.
FR2	Users can submit scans via a GitHub repository URL, which is automatically cloned.
FR3	System recursively walks the repository and submits each source file to the LLM for security audit.
FR4	LLM returns structured JSON findings with severity, line numbers, CWE/OWASP IDs, and code snippet.
FR5	System automatically enriches CRITICAL and HIGH findings with explanation, PoC, and fix code.
FR6	Scan progress is streamed live to the dashboard via Server-Sent Events.
FR7	All scans and issues are persisted in PostgreSQL for historical comparison.
FR8	Dashboard displays severity charts, issue list, Monaco code viewer, and AI Insight Panel.

Table 3.1 — Functional Requirements

3.3 Non-Functional Requirements

- **Performance:** UI response time under two seconds; scan jobs are processed asynchronously without blocking the API.
- **Security:** GitHub OAuth, JWT authentication, Helmet.js HTTP security headers, Zod runtime validation, and Docker sandboxed execution.
- **Scalability:** a stateless REST API supports horizontal scaling; multi-LLM fallback distributes AI load; Prisma connection pooling handles concurrent requests.

- **Reliability:** multi-provider LLM fallback ensures scans complete even under rate limits; graceful error handling marks failed scans with descriptive messages.
- **Usability:** a React dashboard with real-time updates, severity colour coding, and plain-English AI explanations — no security expertise required.

3.4 Process Description

- **User Authentication:** the user initiates the GitHub OAuth flow; Passport.js exchanges the code for an access token; the user profile is stored or updated in PostgreSQL; a JWT is issued for subsequent API calls.
- **Scan Submission:** the user submits a GitHub repository URL. The system validates the input, creates a Scan record in PENDING state, and returns the scanId. The client then subscribes to the SSE stream.
- **Source Acquisition:** the Bull queue worker runs git clone --depth=1 --single-branch into a system temp directory (os.tmpdir()/sast-<scanId>-XXXX). The scan status is updated to CLONING during acquisition and advances to SCANNING once the source is ready.
- **AI Security Scan:** AiScannerService recursively walks the working directory, skipping irrelevant folders (node_modules, .git, dist, build, vendor, __pycache__, .next, coverage) and files above 60 KB. Each qualifying source file is submitted to the LLM provider chain; findings are persisted to PostgreSQL in real time and the Scan record's live counts and risk score are updated after each file batch.
- **Result Persistence:** structured JSON findings are parsed, normalised, and bulk-inserted into PostgreSQL. The risk score is calculated and the Scan record is updated.

- **AI Enrichment:** CRITICAL and HIGH severity findings with aiProcessed = false are sent to the AI enrichment service. Structured insights (explanation, impact, proofOfConcept, remediation, fixCode) are written directly onto the Issue records via Prisma update, and aiProcessed is set to true.
- **Completion**: the scan status is updated to COMPLETED with the elapsed duration and completedAt timestamp. The SSE polling loop emits a final payload with done: true. The working directory is deleted in the queue worker's finally block. The frontend then fetches the full results summary and issue list.

4. SOFTWARE DESIGN

This chapter presents the architectural design, the data flow diagrams, and the UML diagrams that describe the structure and behaviour of the AI Powered SAST Agent. The design is intended to be implementation-neutral so that it can be re-implemented in any modern web stack while preserving the system contracts.

4.1 System Architecture

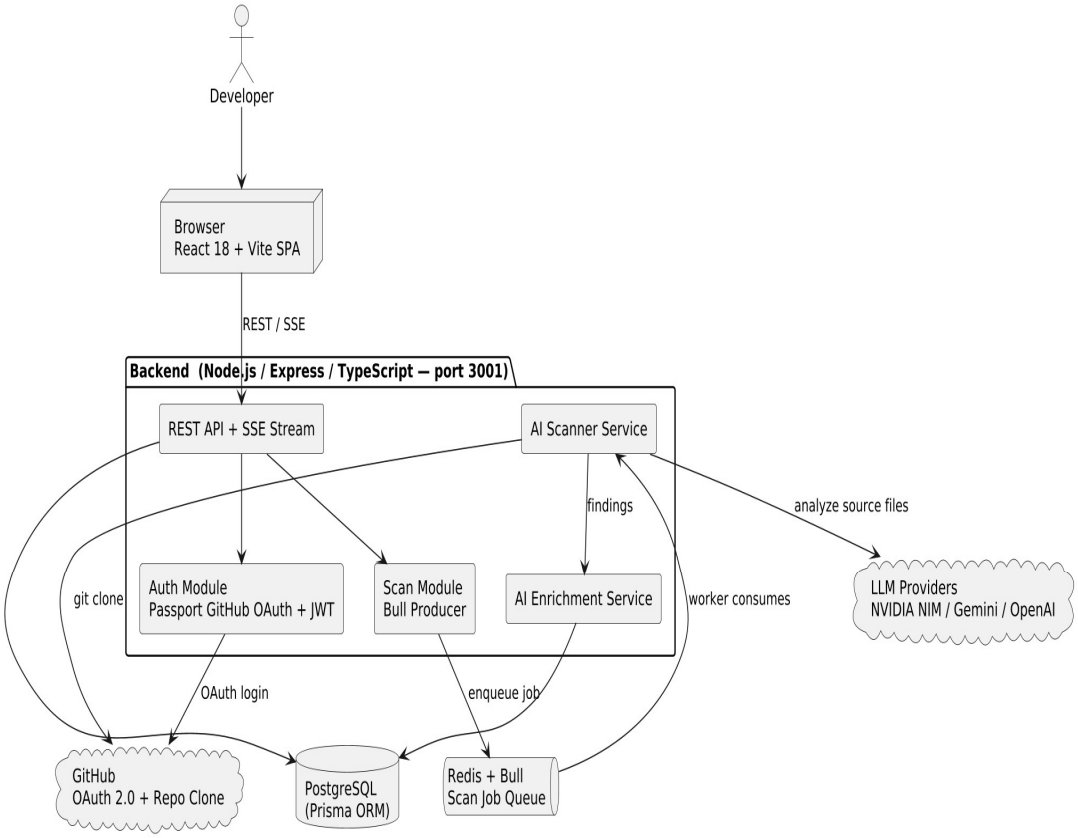


Figure 4.1 — System Architecture

4.2 Data Flow Diagrams

Data Flow Diagrams describe how data moves through the system. Level 0 presents the system as a single process interacting with external entities, while Level 1 decomposes the system into its principal sub-processes.

4.2.1 Level 0 — Context Diagram

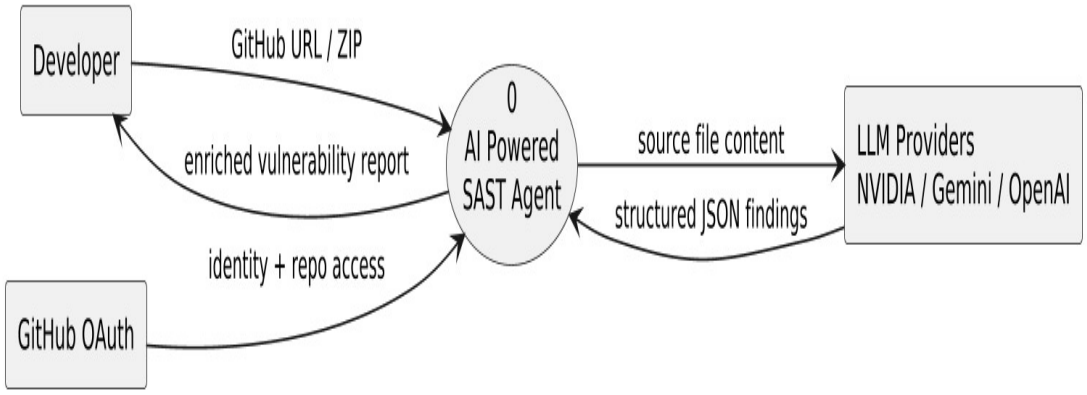


Figure 4.2.1 — Data Flow Diagram, Level 0

4.2.2 Level 1 — System Decomposition

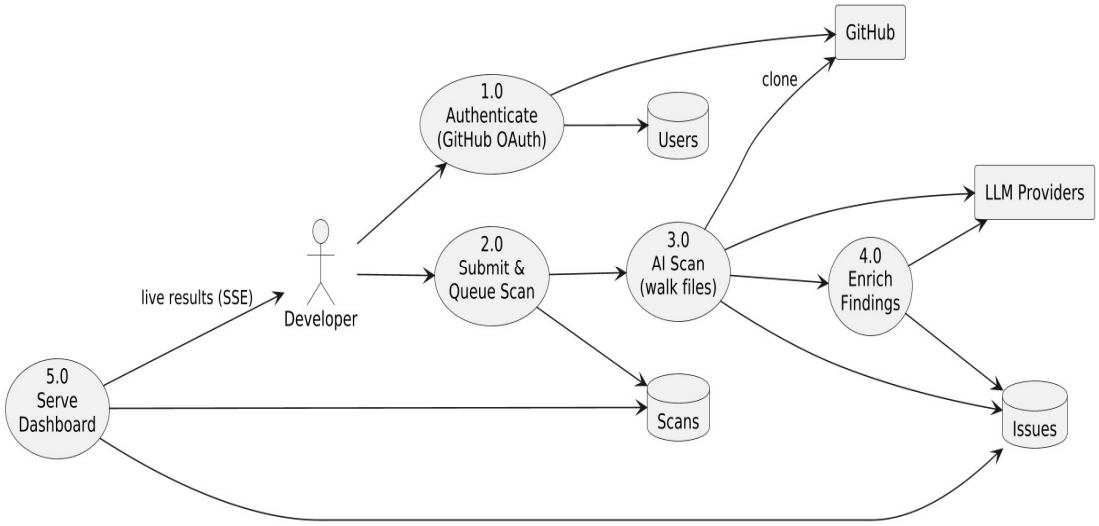


Figure 4.2.2 — Data Flow Diagram, Level 1

4.3 UML Diagrams

The Unified Modelling Language (UML) diagrams below capture the static structure and dynamic behaviour of the system. Use case, activity, class, sequence, collaboration, component, deployment, and state chart diagrams together describe the system from multiple perspectives.

4.3.1 Use Case Diagram

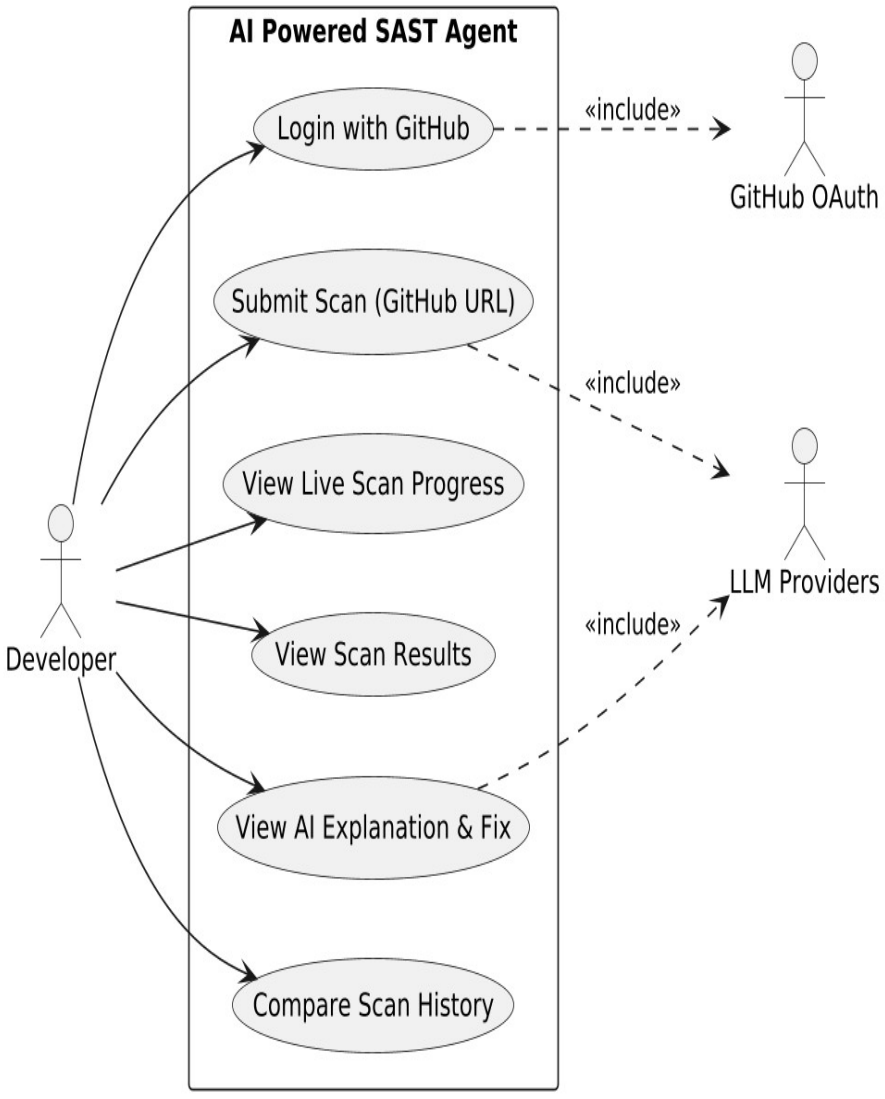


Figure 4.3.1 — Use Case Diagram

4.3.2 Activity Diagram

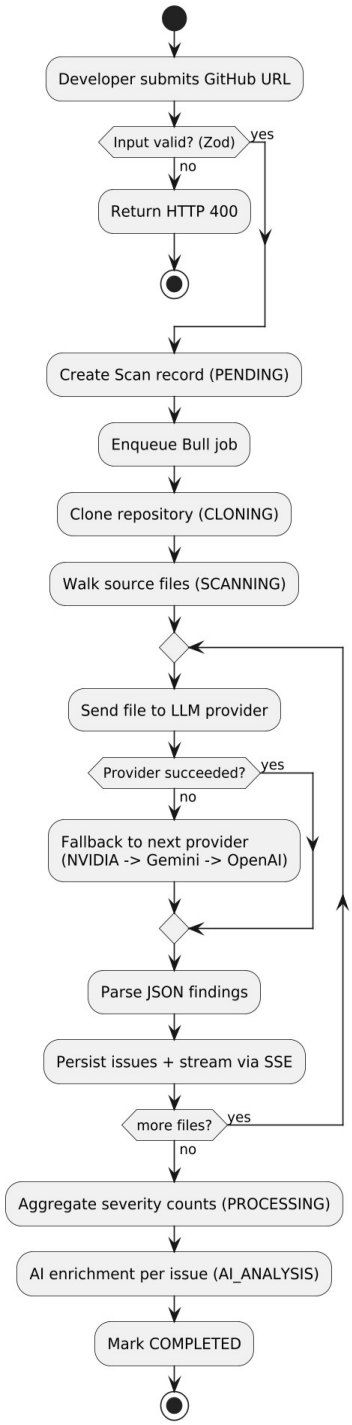


Figure 4.3.2 — Activity Diagram

4.3.3 Class Diagram

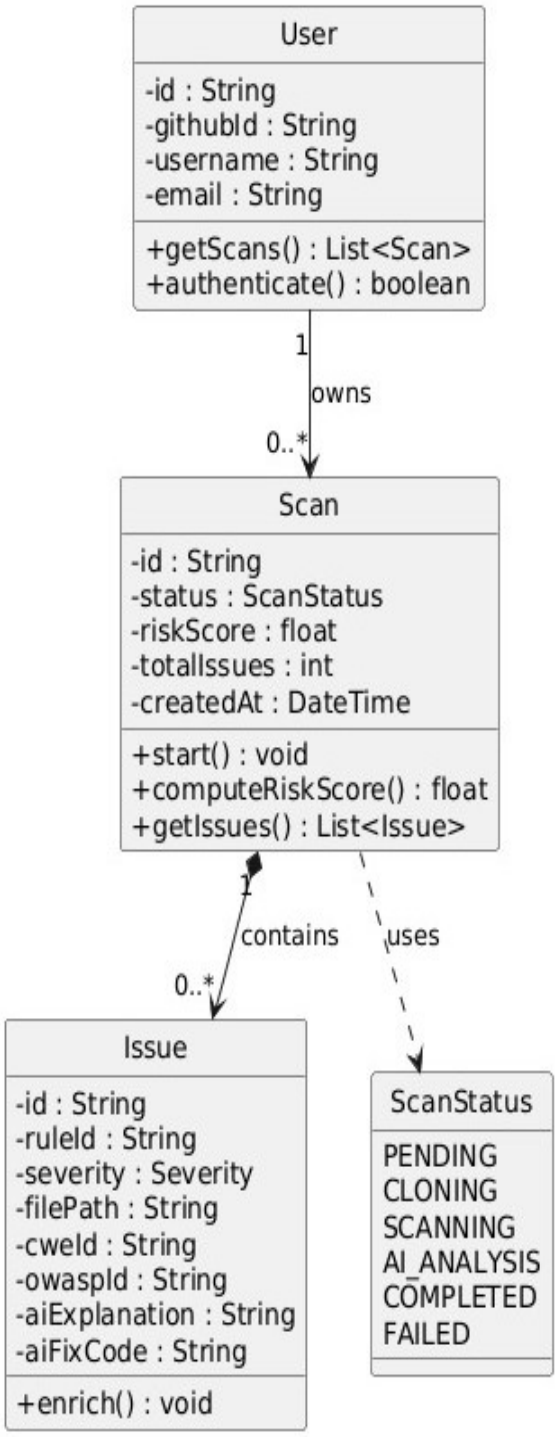


Figure 4.3.3 — Class Diagram

4.3.4 Sequence Diagram

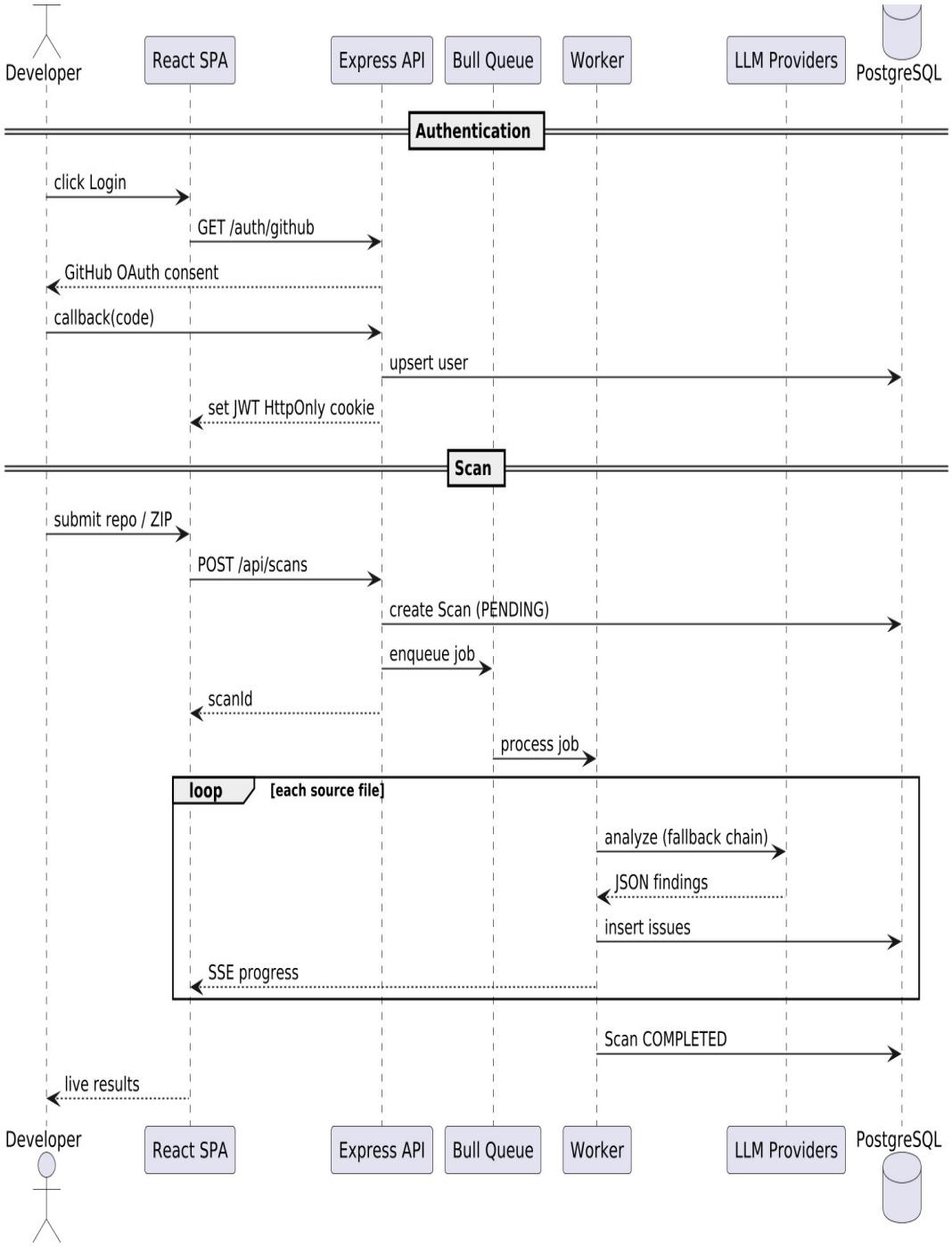


Figure 4.3.4 — Sequence Diagram

4.3.5 Collaboration Diagram

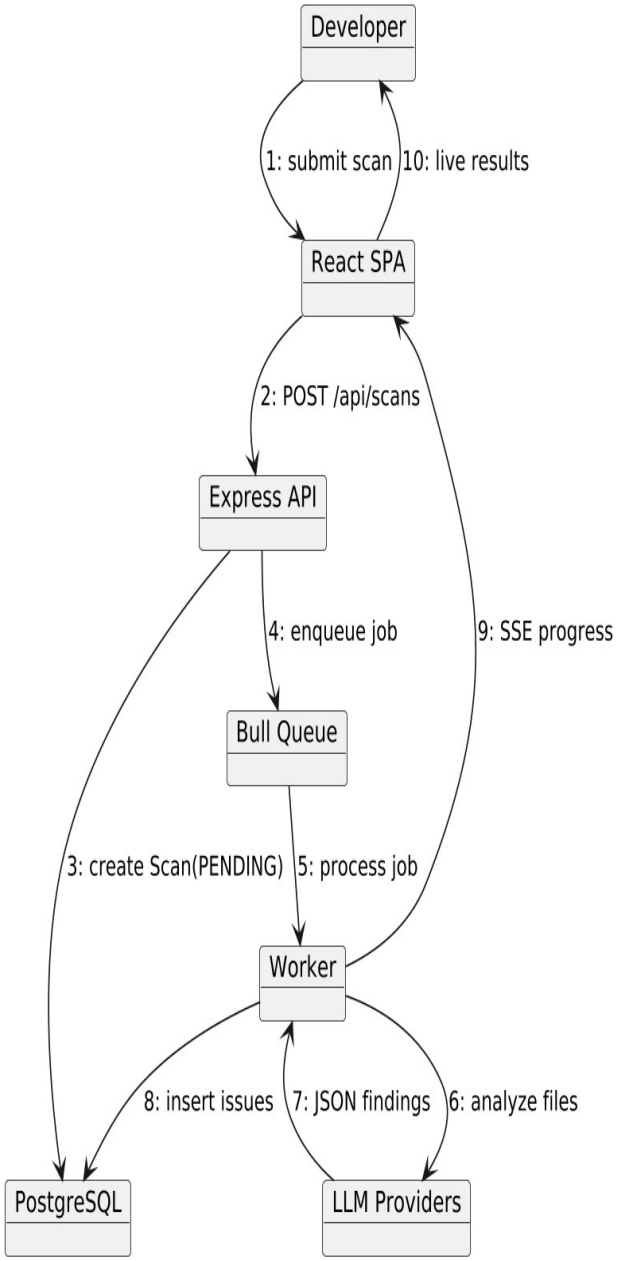


Figure 4.3.5 — Collaboration Diagram

4.3.6 Component Diagram

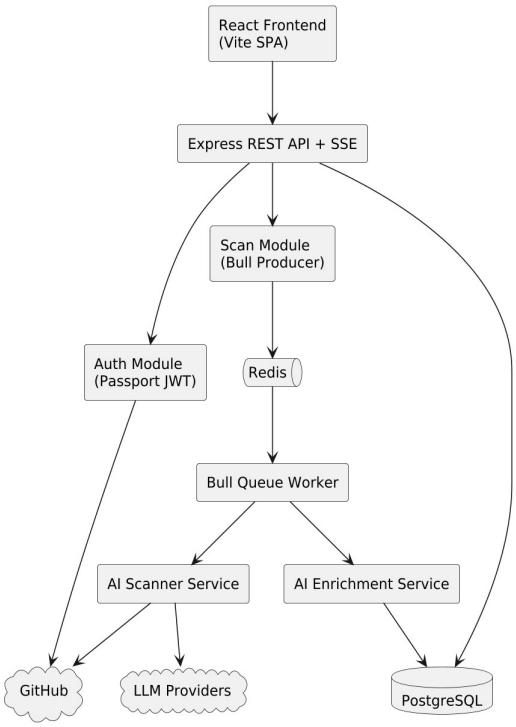


Figure 4.3.6 — Component Diagram

4.3.7 Deployment Diagram

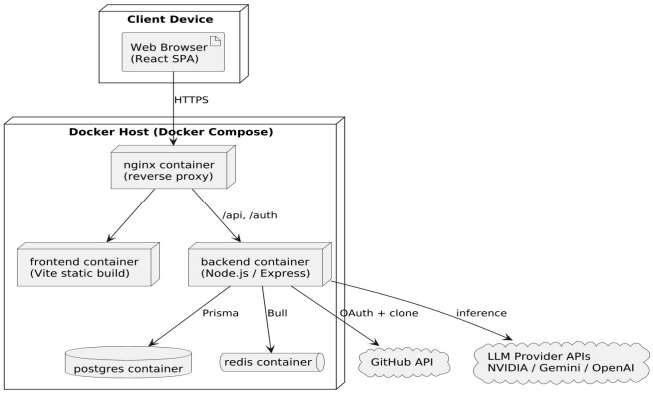


Figure 4.3.7 — Deployment Diagram

4.3.8 State Chart Diagram

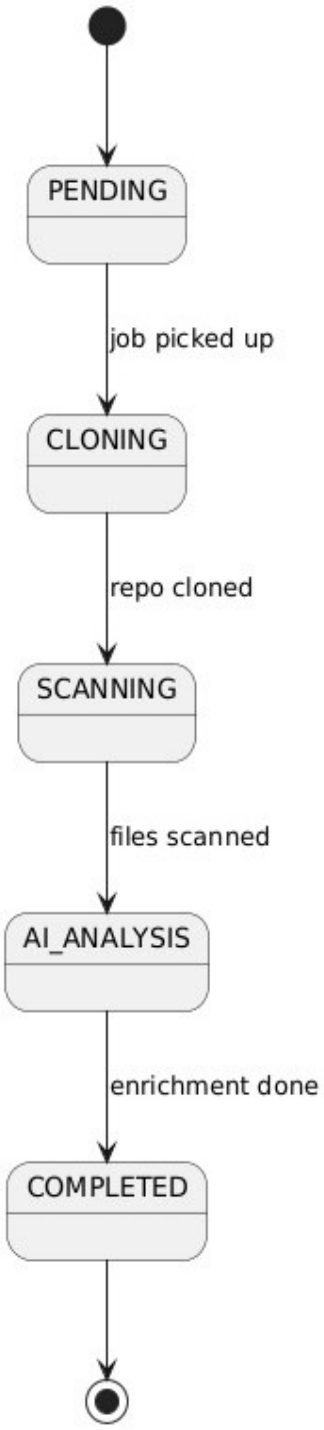


Figure 4.3.8 — State Chart Diagram

5. SOFTWARE AND HARDWARE REQUIREMENTS

To ensure the successful development, deployment, and functioning of the proposed system, the following hardware and software requirements are necessary.

5.1 Hardware Requirements

The system can run on commodity developer hardware. The recommended configuration is required when scanning large repositories or running the full stack (API, frontend, PostgreSQL, Redis) on a single developer workstation.

Component	Specification
Processor	Minimum Dual-Core 2.0 GHz x86-64; Quad-Core 3.0 GHz or higher recommended
RAM	Minimum 8 GB; 16 GB recommended
Storage	Minimum 20 GB free SSD; 50 GB SSD recommended for large repositories
Network	Active broadband internet connection (required for LLM API calls and GitHub cloning)
Display	Minimum 1280 × 720 resolution
Input Devices	Standard keyboard and mouse

Table 5.1 — Hardware Requirements

5.2 Software Requirements

The software stack is fully open-source and cross-platform. All third-party AI providers offer free tiers or generous developer quotas sufficient for the project demonstration.

Component	Specification
Operating System	Windows 10/11, macOS 12 or later, or Ubuntu 22.04 or later
Runtime	Node.js 20 LTS
Coding Language	TypeScript 5.x
Frontend	React 18.x, Vite 5.x, Tailwind CSS 3.x, Recharts 2.x, Monaco Editor
Backend	Express.js 4.x, Prisma ORM 5.x
Database	PostgreSQL 16.x
Job Queue Store	Redis 7.x
AI Providers	NVIDIA NIM (Llama 3.1), Google Gemini 1.5 Flash, OpenAI GPT-4o-mini
Authentication	GitHub OAuth 2.0 via Passport.js, Helmet.js, Zod 3.x
Containerisation	Docker Engine 24+ and Docker Compose
Web Server	Nginx Alpine (reverse proxy)
Browser	Chrome 120+ or Firefox 120+

Table 5.2 — Software Requirements

6. CODING

This chapter presents the key source code listings that implement the core functionality of the AI Powered SAST Agent: the recursive file walker, the multi-LLM service, the scan processor that orchestrates the full lifecycle, the Prisma data model, and the data dictionary describing the principal database tables.

6.1 AI Scanner Service — Recursive File Walker

The AiScannerService walks the cloned or extracted repository directory, identifies source files by extension, skips uninteresting directories (node_modules, .git, dist, build, vendor), and submits each file to the LLM for a security audit.

```
const SUPPORTED =
['.ts','.tsx','.js','.jsx','.py','.go','.java','.rb','.php'];
const SKIP_DIRS  = new
Set(['node_modules','.git','dist','build','vendor']);
export class AiScannerService {
  private ai = new AIService();
  async scanDirectory(dir: string): Promise<RawFinding[]> {
    const files = this.walk(dir);
    const findings: RawFinding[] = [];
    for (const filePath of files) {
      const code    = fs.readFileSync(filePath, 'utf-8');
      const found = await this.ai.auditFile(relPath, code);
      findings.push(...found);
    }
    return findings;
  }

  private walk(dir: string): string[] {
    const out: string[] = [];
    for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
      if (SKIP_DIRS.has(e.name)) continue;
      const full = path.join(dir, e.name);
      if (e.isDirectory())
        out.push(...this.walk(full));
      else if (SUPPORTED.includes(path.extname(e.name)))
        out.push(full);
    }
    return out;
  }
}
```

6.2 AI Service — Multi-LLM Fallback

The AIService class encapsulates the multi-provider LLM pipeline. Providers are registered at startup based on which API keys are present in the environment (NVIDIA_API_KEY, GEMINI_API_KEY, OPENAI_API_KEY), establishing a priority-ordered list. The AiScannerService invokes providers sequentially via scanFileWithFallback: if the first provider raises any exception, the next provider in the chain is attempted automatically until one succeeds or all are exhausted.

```
const AUDIT_PROMPT = `You are an expert security engineer.
Analyse the provided source code for security vulnerabilities.
Return ONLY a JSON array with this shape:
[{ "ruleId": string, "title": string,
   "severity": "CRITICAL"|"HIGH"|"MEDIUM"|"LOW",
   "category": string, "lineStart": number, "lineEnd": number,
   "codeSnippet": string, "cweId": string, "owaspId": string }]
Return [] if no vulnerabilities are found.`;

export class AIService {
  async auditFile(path: string, code: string): Promise<RawFinding[]>
{
    const prompt = `File: ${path}\n\n${code}`;
    try      { return await this.callGemini(prompt); }
    catch(e: any) {
      if (e.status === 429) return await this.callOpenAI(prompt);
      throw e;
    }
  }

  private async callGemini(prompt: string) {
    const model  = gemini.getGenerativeModel({ model: 'gemini-1.5-
flash' });
    const result = await model.generateContent({
      systemInstruction: AUDIT_PROMPT,
      contents: [{ role: 'user', parts: [{ text: prompt }] }],
      generationConfig: { maxOutputTokens: 1200, temperature: 0.1 },
    });
    return JSON.parse(result.response.text());
  }
}
```

6.3 Scan Processor — Full Lifecycle Orchestration

The Bull queue worker function is the top-level orchestrator. It transitions the scan through PENDING, CLONING, SCANNING, AI_ANALYSIS, and either COMPLETED or FAILED states. It is responsible for source acquisition, invoking AiScannerService, persisting findings in real time, triggering AI enrichment for CRITICAL and HIGH severity issues, updating aggregate counts and risk score on the Scan record, and cleaning up the temporary working directory in a finally block.

```
export async function processScan(
  scanId: string, ref: string, emit: Emitter
) {
  await prisma.scan.update({
    where: { id: scanId }, data: { status: 'CLONING' }
  });
  emit({ event: 'status', data: 'RUNNING' });

  const dir = path.join(TMP, scanId);
  try {
    // 1. Acquire source code
    await git.clone(['--depth=1', ref, dir]);

    // 2. AI-native security scan
    const scanner  = new AiScannerService();
    const findings = await scanner.scanDirectory(dir);
    await prisma.issue.createMany({
      data: findings.map(f => ({ ...f, scanId }))
    });
    emit({ event: 'progress', data: `${findings.length} issues
found` });

    // 3. AI enrichment (explanation, PoC, fix)
    const ai = new AIService();
    for (const iss of findings.filter(
      f => ['CRITICAL','HIGH'].includes(f.severity)
    )) {
      const insight = await ai.enrich(iss);
      await prisma.issue.update({
        where: { id: iss.id },
        data: { aiExplanation: insight.explanation, aiImpact:
insight.impact,
          aiRemediation: insight.remediation, aiProcessed: true },
      });
```

```
    }

    // 4. Finalise
    const score = calcRisk(findings);
    await prisma.scan.update({
      where: { id: scanId },
      data: { status: 'COMPLETED', riskScore: score }
    });
    emit({ event: 'status', data: 'COMPLETED' });

  } catch (err: any) {
    await prisma.scan.update({
      where: { id: scanId },
      data: { status: 'FAILED', errorMsg: err.message }
    });
    emit({ event: 'status', data: 'FAILED' });
  } finally {
    await fs.rm(dir, { recursive: true, force: true });
  }
}
```

6.4 Prisma Schema — Database Models

The Prisma schema declares the relational data model. The Scan and Issue models capture the core domain entities. AI enrichment data is stored directly on the Issue model as nullable text fields (aiExplanation, aiImpact, aiProofOfConcept, aiRemediation, aiFixCode) with a boolean aiProcessed flag — no separate AiInsight table is required. Enums encode the full lifecycle states and severity levels.

```
model Scan {
  id          String     @id @default(cuid())
  userId      String
  status      ScanStatus @default(PENDING)
  totalIssues Int        @default(0)
  riskScore   Float      @default(0)
  user        User       @relation(fields:[userId], references:[id])
  issues      Issue[]
  createdAt   DateTime   @default(now())
}
model Issue {
  id          String     @id @default(cuid())
  scanId      String
  ruleId      String
  title       String
```

```
  severity    Severity
  filePath    String
  lineStart   Int
  codeSnippet String
  cweId       String?
  owaspId     String?
  scan        Scan       @relation(fields:[scanId], references:[id])
  aiProcessed  Boolean  @default(false)
}

enum ScanStatus { PENDING CLONING SCANNING PROCESSING AI_ANALYSIS
COMPLETED FAILED }
enum Severity   { CRITICAL HIGH MEDIUM LOW WARNING INFO }
```

6.5 Data Dictionary

The data dictionary describes the structure of the two principal database tables: scans and issues. AI enrichment columns are part of the issues table itself; there is no separate ai_insights table.

6.5.1 Scans Table

Purpose: stores all scan records, their status lifecycle, input type, computed risk score, and aggregate issue counts.

Field Name	Data Type	Max Length	Description
id	String (cuid)	—	Primary key — unique scan identifier
status	ScanStatus enum	—	PENDING / CLONING / SCANNING / AI_ANALYSIS / COMPLETED / FAILED
totalIssues	Int	—	Total vulnerabilities found in this scan
riskScore	Float	—	Weighted score: Critical×10 + High×5 + Med×2 + Low×1
createdAt	DateTime	—	Scan submission timestamp

Table 6.1 — Data Dictionary, scans Table

6.5.2 Issues Table

Purpose: stores individual vulnerability findings produced by the AI Scan Engine.

Field Name	Data Type	Max Length	Description
id	String (cuid)	—	Primary key
scanId	String FK	—	Reference to scans table
ruleId	String	200	LLM-generated rule identifier (e.g. SQL_INJECTION_001)
severity	Severity enum	—	CRITICAL / HIGH / MEDIUM / LOW / WARNING
filePath	String	500	Relative path to the vulnerable file
codeSnippet	String	2000	Extracted vulnerable code fragment
cweId	String	50	CWE identifier (e.g. CWE-89)
owaspId	String	50	OWASP Top-10 category (e.g. A03:2021)

Table 6.2 — Data Dictionary, issues Table

7. TESTING

To verify the reliability and functional correctness of the platform, a structured testing process was carried out across three complementary levels:

- **Unit Testing:** focuses on individual service methods and utility functions, tested in isolation using Vitest with mocked dependencies.
- **Integration Testing:** ensures that different modules (Express API, AiScannerService, PostgreSQL, Redis) interact correctly.
- **Functional Testing:** validates the complete system functionality from a developer's perspective.

Each test case is documented below with its objective, input, and the expected and actual outputs.

7.1 Unit Testing Table

Test Case ID	Test Description	Input Data	Expected Output	Status
TC_U1	AI JSON response parser	Valid JSON findings array	Typed Issue[] returned	Pass
TC_U5	Zod schema validation	Invalid GitHub URL	Validation error returned	Pass

Table 7.1 — Unit Testing Table

7.2 Integration Testing Table

Test Case ID	Test Description	Modules	Input Data	Expected Output	St.
TC_I1	POST /api/scans/github	API + DB	Valid GitHub	HTTP 202, Scan PENDING	Pass
TC_I2	GET /api/scans/:id	API + DB	Valid scanId	Scan data with issues	Pass

Table 7.2 — Integration Testing Table

7.3 Functional Testing Table

Test Case ID	Test Description	Scenario	Expected Output	Status
TC_F1	Full scan flow	Submit URL → SSE → dashboard	End-to-end completes	Pass
TC_F4	AI enrichment	CRITICAL issue found	Issue AI enrichment fields (aiExplanation, aiImpact, aiProofOfConcept, aiRemediation, aiFixCode) populated on Issue record	Pass
TC_F5	Scan history comparison	Select 2 scans	New/resolved issues highlighted	Pass

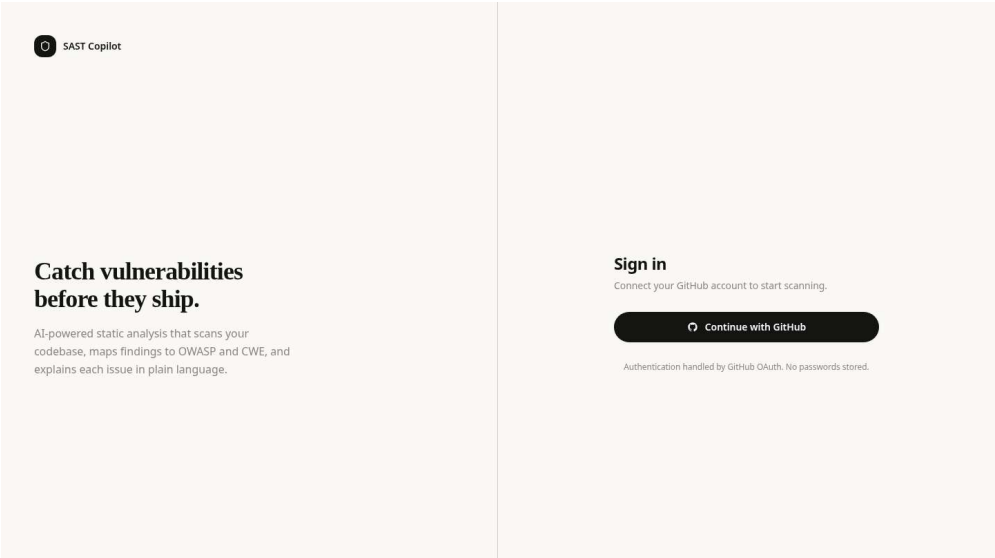
Table 7.3 — Functional Testing Table

8. OUTPUT SCREENS

This chapter presents four representative output screens of the AI Powered SAST Agent platform, covering the complete developer workflow from authentication through final results. Each section describes the screen content and the key UI elements visible at that stage.

8.1 GitHub OAuth Login Page

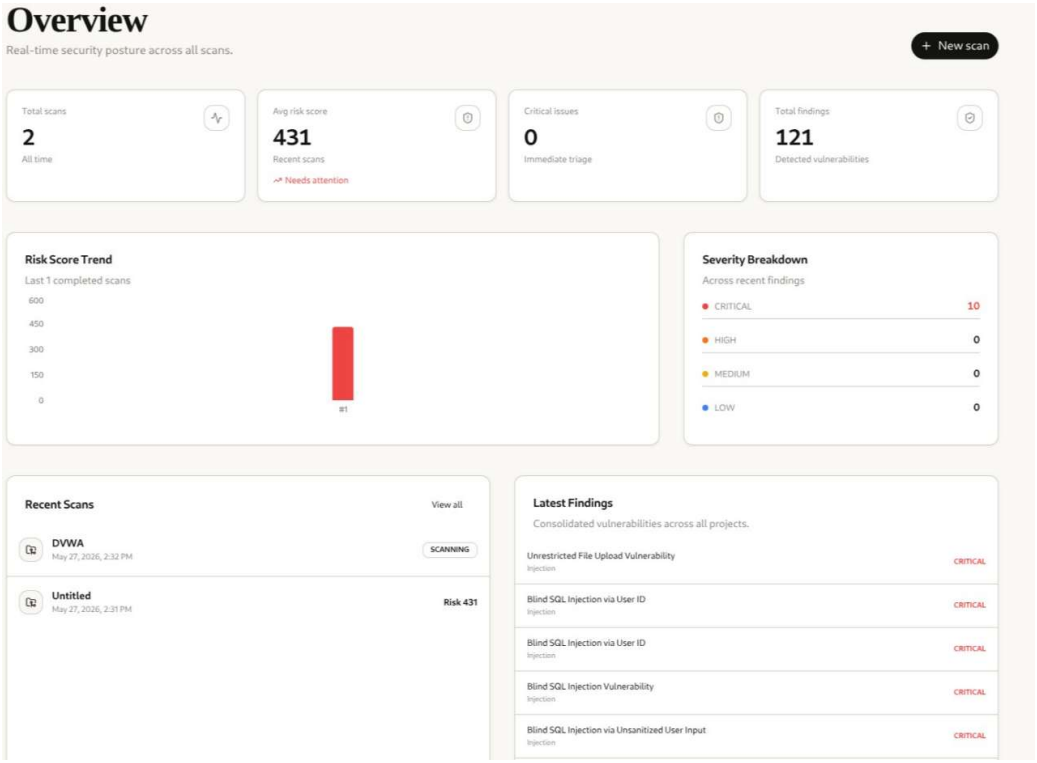
The login page displays a single “Sign in with GitHub” button along with the product branding. No password fields are present, since authentication is fully delegated to GitHub via OAuth 2.0.



Screenshot 8.1 — GitHub OAuth Login Page

8.2 Main Dashboard Overview

The home dashboard summarises the developer's security posture at a glance: total scans, total issues, and the average risk score across all scans.



Screenshot 8.2 — Main Dashboard Overview

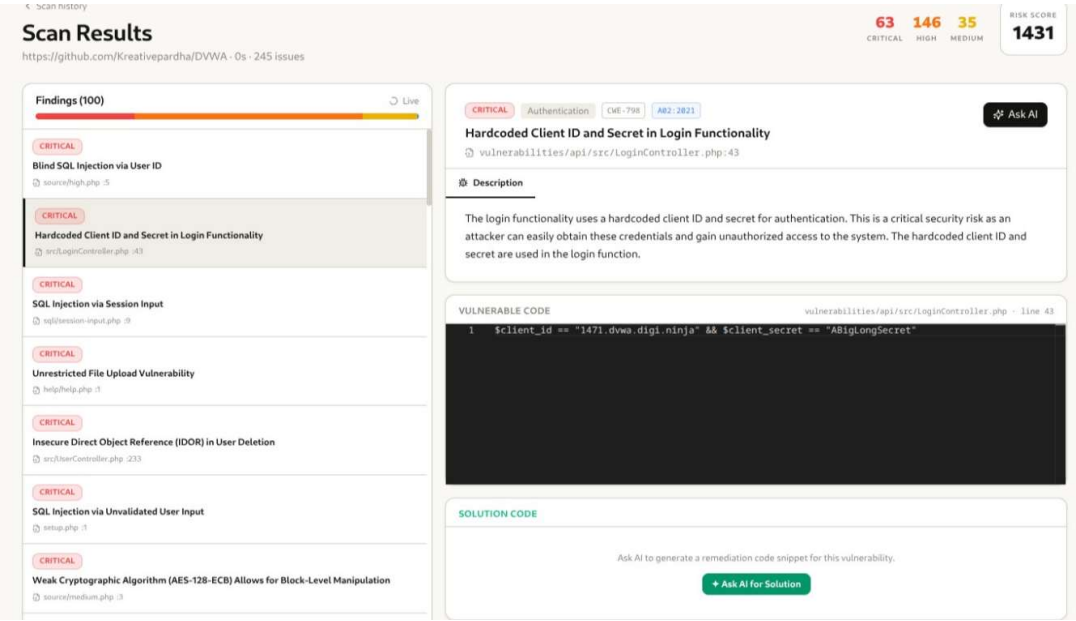
8.3 New Scan Submission Form

The scan submission form accepts a GitHub repository URL. The developer pastes the URL and the backend validates its format using Zod before accepting the submission, returning HTTP 201 with the new scanId on success.

Screenshot 8.3 — New Scan Submission Form

8.4 Scan Results Dashboard

The scan results dashboard presents a colour-coded donut chart showing the distribution of CRITICAL, HIGH, MEDIUM, and LOW severity findings. The findings list on the left shows each vulnerability with its severity badge, category tag, file path, and line number. The risk score, computed as Critical×10 + High×5 + Medium×2 + Low×1, is displayed prominently in the header. Selecting any finding loads the AI Insight Panel on the right with the plain-English explanation, proof-of-concept attack scenario, and ready-to-apply code fix.



Screenshot 8.4 — Scan Results Dashboard

9. CONCLUSION

9.1 Project Conclusion

This project addresses the challenge of developer security by using Large Language Models as the primary scanning engine. By submitting source files directly to LLMs with structured audit prompts, the system produces context-aware findings that rule-based engines cannot — including logic-level flaws, insecure control flow, and cross-framework patterns.

The AI Powered SAST Agent met all its stated objectives: a fully containerised full-stack (TypeScript Node.js/Express, React 18, PostgreSQL/Prisma, Redis-backed Bull queues, Docker Compose) with a resilient three-provider pipeline (NVIDIA NIM to Gemini 1.5 Flash to GPT-4o-mini) that completes scans even when a provider is unavailable, and an enrichment layer that writes plain-English explanations, proof-of-concept scenarios, and code-level fixes onto every CRITICAL and HIGH finding.

Overall, the project demonstrates a practical, extensible architecture for AI-native application security testing, showing how LLMs can replace — not merely augment — traditional rule-based scanners.

9.2 Further Enhancement

Several enhancements have been identified that would extend the platform's utility in industry settings:

- **Browser Extension for Real-Time Scanning:** a Chrome and Firefox browser extension that integrates the AI scanning engine. The extension would automatically analyse JavaScript source code loaded in the browser and alert developers to potential vulnerabilities as they browse or test their applications.
- **CI/CD Pipeline Integration:** provide a GitHub Action and GitLab CI job definition that triggers AI SAST scans on every pull request, blocking

merges on CRITICAL severity findings and posting AI-generated fix suggestions directly on vulnerable code lines using the GitHub Checks API.

- **Fine-Tuned Security LLM:** fine-tune an open-source model such as CodeLlama on a verified vulnerability-and-fix dataset to improve remediation accuracy and reduce hallucinations, particularly for less common vulnerability classes like SSRF and XXE injection.
- **Compliance Reporting:** generate compliance-mapped reports aligned to PCI-DSS, HIPAA, and OWASP ASVS frameworks, enabling use in regulated industries and enterprise security audits.
- **Dynamic Application Security Testing (DAST):** extend the platform with a dynamic testing layer that complements the static analysis with runtime checks, providing fuller coverage of vulnerabilities that only manifest at execution time.

10. BIBLIOGRAPHY

The following references were consulted during the design, implementation, and documentation of this project. They follow the format: Author Name, “Title of the book or paper”, Publisher name, Volume, Year.

1. Google DeepMind, “Gemini: A Family of Highly Capable Multimodal Models”, Google AI, 2024. https://ai.google.dev/gemini-api/docs
2. OpenAI, “GPT-4o Technical Report”, OpenAI, 2024. https://openai.com/research/gpt-4o
3. NVIDIA, “NVIDIA NIM: Optimised LLM Inference Microservices”, NVIDIA Developer, 2024. https://developer.nvidia.com/nim
4. OWASP Foundation, “OWASP Top 10 for Large Language Model Applications 2025”, OWASP, 2025. https://genai.owasp.org/llm-top-10/
5. B. Livshits and M. Lam, “Finding Security Vulnerabilities in Java Applications with Static Analysis”, Proceedings of the 14th USENIX Security Symposium, pp. 271–286, 2005.
6. B. Johnson et al., “Why Don’t Software Developers Use Static Analysis Tools to Find Bugs?”, Proceedings of the 35th International Conference on Software Engineering, pp. 672–681, IEEE, 2013.
7. C. Tony et al., “LLMSecEval: A Dataset of Natural Language Prompts for Security Evaluations”, arXiv:2303.09384, 2023.
8. E. Yildirim et al., “ChatGPT vs. Static Analysis: Evaluating LLMs for OWASP API Security Vulnerability Detection”, arXiv:2412.09734, 2024.
9. Prisma Team, “Prisma ORM Documentation”, Prisma, 2024. https://www.prisma.io/docs
10. Microsoft, “Monaco Editor: The Code Editor that Powers VS Code”, Microsoft, 2024. https://github.com/microsoft/monaco-editor.